Project Title

Delivery Route Optimizer

Project Report submitted for the partial fulfilment of grade for the course

Data Structures and Algorithms -3 (25CS2103E)

in

A.Y. 2026-2027

Submitted by
R Radhakrishna (2520090063)
R Jagadeesh (2520090030)

Team Number: 1

Section: 11

Under the guidance of

Mrs. Ch. Anitha

Asst. Professor, CS&IT



Hyderabad-500090, Telangana, India.

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



DECLARATION

The Mini Project Report entitled "**Delivery Route Optimizer**" is a record of Bonafide work of **R Radhakrishna – 2520090063, R Jagadeesh – 2520090030** submitted in partial fulfillment for the award of B. Tech in Computer Science and Information Technology to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

R Radhakrishna – 2520090063

R Jagadeesh – 2520090030

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



CERTIFICATE

This is to certify that the mini project based report entitled "**Delivery Route Optimizer**" is a bonafide work done and submitted by **R Radhakrishna – 2520090063, R Jagadeesh – 2520090030** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science and Information Technology, K L (Deemed to be University), during the academic year **2026-2027**.

Signature of the Supervisor

Signature of the HOD **Signature of the External Examiner**

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. We wish to express our sincere thanks to all those who have assisted us in one way or the other for the completion of our project.

Our greatest appreciation to our guide, **Mrs. Ch. Anitha**, Asst. Professor, Department of Computer Science and Information Technology, which cannot be expressed in words for her tremendous support, encouragement and guidance for this project.

We express our gratitude to **Dr. Khaja Shareef**, Head of the Department of Computer Science and Information Technology, for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff, and also those who have assisted us directly or indirectly for the successful completion of this project. Finally, we sincerely thank our parents, friends and classmates for their kind help and cooperation during our work.

R Radhakrishna – 2520090063

R Jagadeesh – 2520090030

TABLE OF CONTENTS

S.No.	Contents	Page no.
1.	Abstract	6
2.	Introduction	7
3.	Literature survey	8
4.	Problem Statement and Objectives	9
5.	Hardware and Software requirements	10
6.	Implementation	11
7.	Results & Discussion	14
8.	Conclusion & Future Scope	16
9.	References	17

1.ABSTRACT

The Delivery Route Optimizer is a Java-based project developed to improve the efficiency of delivery operations by identifying the shortest and most cost-effective routes between multiple locations. The system applies Data Structures and Algorithms (DSA) concepts, particularly graph representations and shortest path algorithms such as Dijkstra's Algorithm, to calculate the optimal delivery path. Users can enter delivery locations, distances, and destination details, and the application processes this information to generate the best possible route while minimizing travel distance and delivery time. The project demonstrates the practical application of graphs, priority queues, and efficient searching techniques in solving real-world logistics problems. It also provides a simple and user-friendly interface for entering route information and displaying optimized results. By reducing unnecessary travel and improving route planning, the system helps save time, fuel, and operational costs. This project enhances understanding of algorithm design, graph data structures, and optimization techniques while showcasing how DSA can be applied to modern transportation and logistics challenges. The Delivery Route Optimizer serves as an educational and practical solution for efficient route management.

Team Name: Team 1

Team Logo (if any): Not applicable

Team Members:

- R Radhakrishna, 2520090063
- R Jagadeesh, 2520090030

2. INTRODUCTION

Efficient delivery routing is critical for modern logistics, e-commerce and transportation services. Every day, couriers, food-delivery partners and supply-chain operators must visit many locations, and the order and roads they choose directly decide how much time, fuel and money each trip consumes. Manual route planning is slow, error-prone and rarely gives the most cost-effective path, especially as the number of locations and possible connections grows.

The Delivery Route Optimizer addresses this problem by automating route selection with Data Structures and Algorithms. The delivery network is modeled as a weighted graph in which locations are nodes and the distances between them are edges. Dijkstra's Algorithm, implemented with a Priority Queue (Min-Heap), then computes the shortest path from a source location such as a warehouse to every delivery destination. The goal is to determine the shortest path between a source and multiple delivery destinations, so that unnecessary travel is avoided.

The project is built in Java, combining a strong algorithmic foundation with a practical, usable application. It uses the Java Collections Framework (PriorityQueue, HashMap and ArrayList) for the underlying data structures and gives the user a simple interface to enter locations, distances and destinations and to view the optimized route together with its total distance.

The rest of this report is organized as follows: Section 3 surveys related algorithms and systems; Section 4 states the problem and objectives; Section 5 lists the hardware and software requirements; Section 6 describes the implementation; Section 7 presents the results and discussion; and Section 8 gives the conclusion and future scope, followed by the references.

3. LITERATURE SURVEY

1. Dijkstra's Algorithm (1959). Edsger W. Dijkstra proposed a greedy algorithm that finds the shortest path from a single source node to all other nodes of a graph with non-negative edge weights. It repeatedly selects the unvisited node with the smallest known distance and relaxes its outgoing edges. It is the foundation of most routing software and is the algorithm used in this project [1][2].

2. Priority-Queue (Min-Heap) implementation. A simple array-based Dijkstra runs in $O(V^2)$ time, whereas storing candidate nodes in a binary-heap priority queue reduces this to $O((V + E) \log V)$. Java's PriorityQueue class provides this heap, which is why the project uses it to improve the running time considerably on larger delivery networks [2][3].

3. Bellman-Ford Algorithm. Bellman-Ford also computes single-source shortest paths but can handle graphs with negative edge weights and detect negative cycles. It relaxes all edges V-1 times, which makes it slower ($O(V \cdot E)$) than Dijkstra's Algorithm. Since road distances are never negative, Dijkstra's Algorithm is the better fit for delivery routing [4].

4. A* Search Algorithm. A* extends Dijkstra's Algorithm with a heuristic estimate of the remaining distance to the goal, so it explores far fewer nodes when a single destination is required. It is widely used in GPS, navigation systems and games, and is identified as a natural enhancement for this project [5].

5. Vehicle Routing Problem (VRP). The VRP generalizes route planning to a fleet of vehicles serving many customers under capacity and time constraints. Exact solutions are computationally hard, so heuristic and metaheuristic solvers are used in practice. This is a larger problem than a single-source shortest path and is kept as future scope of this project [6].

6. Travelling Salesman Problem (TSP). TSP asks for the shortest tour that visits every location exactly once and returns to the start. It is NP-hard, and shortest-path computations between all pairs of locations are a basic building block of its heuristics. Our system solves the related but simpler shortest-path-to-each-destination problem [7].

7. Google OR-Tools Routing Library. Google's open-source OR-Tools provides production-grade solvers for vehicle routing, time windows and capacity constraints. It shows how graph-based shortest-path techniques underpin commercial logistics software, and serves as a benchmark for the features that can be added to a student-level optimizer [8].

8. Commercial maps and delivery applications. Services such as Google Maps and delivery apps model road networks as graphs and rely on Dijkstra-style and A*-style shortest-path techniques, enhanced with live traffic data. Existing literature therefore supports graphs and shortest-path algorithms as an effective foundation for route optimization [2][8].

4. Problem Statement and Objectives

Problem Statement: To design and implement a Java-based Delivery Route Optimizer that finds the shortest and most cost-effective route from a source to multiple delivery destinations.

Problem Description: Manual and unplanned delivery routing leads to wasted travel distance, higher fuel usage and increased delivery time and cost. A graph-based approach is needed to compute optimal routes automatically.

Objectives:

- Design a graph-based model to represent delivery locations and routes.
- Implement Dijkstra's Algorithm using a priority queue to compute the shortest path efficiently.
- Build a simple interface to input locations, distances and destinations.
- Minimize total travel distance and overall delivery time.
- Display a clear, optimized delivery route as the final output.

5. HARDWARE AND SOFTWARE REQUIREMENTS

Hardware Requirements:

- Processor: Intel i3 or above
- RAM: 4 GB minimum (8 GB recommended)
- Hard Disk: 500 MB free space
- Standard keyboard, mouse and monitor

Software Requirements:

- Operating System: Windows / Linux / macOS
- Programming Language: Java (JDK 8 or above)
- IDE: Eclipse / IntelliJ IDEA / NetBeans
- Java Collections Framework (PriorityQueue, HashMap, ArrayList)

6. IMPLEMENTATION

The system is divided into three modules. The workflow is shown in Fig. 1: the user's locations, distances and destinations are used to build a weighted graph, Dijkstra's Algorithm is applied with a priority queue, and the optimized route with its total distance is displayed.

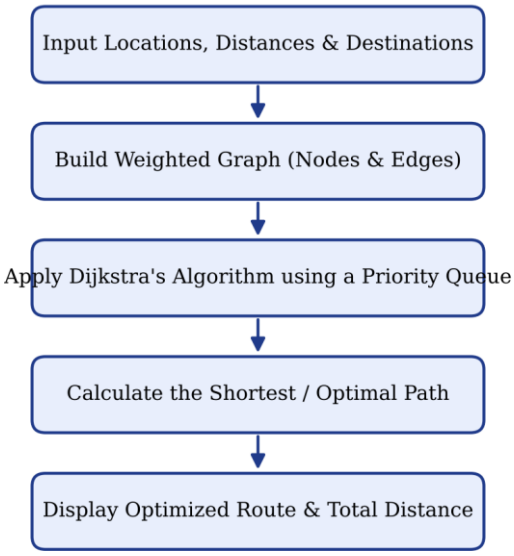


Fig. 1: Flow chart of the Delivery Route Optimizer

Module 1: Graph Representation (Graph.java)

Stores locations as nodes and routes as weighted edges using an adjacency list. A HashMap maps each location name to an integer id, and routes are added in both directions because roads are bidirectional.

```
import java.util.*;

public class Graph {
    static class Edge {
        int to; int weight;
        Edge(int to, int weight) { this.to = to; this.weight = weight; }
    }

    private final Map<String, Integer> index = new LinkedHashMap<>();
    private final List<String> names = new ArrayList<>();
    private final List<List<Edge>> adj = new ArrayList<>();

    public int addLocation(String name) {
        Integer id = index.get(name);
        if (id != null) return id;
        index.put(name, names.size());
        names.add(name);
        adj.add(new ArrayList<>());
        return names.size() - 1;
    }
```

```
    public void addRoute(String a, String b, int km) {
        int u = addLocation(a), v = addLocation(b);
        adj.get(u).add(new Edge(v, km));
        adj.get(v).add(new Edge(u, km));
    }

    public int id(String name) { return index.getOrDefault(name, -1); }
    public String name(int id) { return names.get(id); }
    public int size() { return names.size(); }
    public List<Edge> neighbours(int u) { return adj.get(u); }
}
```

Module 2: Dijkstra's Algorithm (Dijkstra.java)

Computes the shortest distance from the source to every location using a PriorityQueue (min-heap) ordered by distance, and records each node's parent so that the route can be reconstructed. Time complexity is O((V + E) log V).

```
import java.util.*;

public class Dijkstra {
    public final int[] dist;
    public final int[] parent;

    public Dijkstra(Graph g, int source) {
        int n = g.size();
        dist = new int[n];
        parent = new int[n];
        Arrays.fill(dist, Integer.MAX_VALUE);
        Arrays.fill(parent, -1);
        dist[source] = 0;

        PriorityQueue<int[]> pq = new PriorityQueue<>(Comparator.comparingInt(a -> a[1]));
        pq.add(new int[]{source, 0});

        while (!pq.isEmpty()) {
            int[] cur = pq.poll();
            int u = cur[0];
            if (cur[1] > dist[u]) continue;            // stale entry
            for (Graph.Edge e : g.neighbours(u)) {
                int nd = dist[u] + e.weight;
                if (nd < dist[e.to]) {
                    dist[e.to] = nd;
                    parent[e.to] = u;
                    pq.add(new int[]{e.to, nd});
                }
            }
        }
    }

    public List<Integer> pathTo(int target) {
        LinkedList<Integer> path = new LinkedList<>();
        if (dist[target] == Integer.MAX_VALUE) return path;
        for (int v = target; v != -1; v = parent[v]) path.addFirst(v);
        return path;
    }
}
```

Module 3: User Interface and Route Output (RouteOptimizer.java)

Reads the routes, the source location and the delivery destinations from the user, runs Dijkstra's Algorithm once from the source, and prints the optimized route and distance for every destination along with the total distance.

```
import java.util.*;

public class RouteOptimizer {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Graph g = new Graph();

        System.out.print("Enter number of routes: ");
        int m = Integer.parseInt(sc.nextLine().trim());
        System.out.println("Enter each route as: <location1> <location2> <distance_km>");
        for (int i = 0; i < m; i++) {
            String[] t = sc.nextLine().trim().split("\\s+");
            g.addRoute(t[0], t[1], Integer.parseInt(t[2]));
        }

        System.out.print("Enter source (warehouse): ");
        String src = sc.nextLine().trim();
        if (g.id(src) < 0) { System.out.println("Unknown source location."); return; }

        System.out.print("Enter delivery destinations (space separated): ");
        String[] dests = sc.nextLine().trim().split("\\s+");

        Dijkstra d = new Dijkstra(g, g.id(src));

        System.out.println("\n========== OPTIMIZED DELIVERY ROUTES ==========");
        int total = 0;
        for (String dest : dests) {
            int t = g.id(dest);
            if (t < 0) { System.out.println(dest + " : location not found"); continue; }
            List<Integer> p = d.pathTo(t);
            if (p.isEmpty()) { System.out.println(dest + " : not reachable"); continue; }
            StringBuilder sb = new StringBuilder();
            for (int i = 0; i < p.size(); i++) {
                if (i > 0) sb.append(" -> ");
                sb.append(g.name(p.get(i)));
            }
            System.out.printf("%-12s: %s  (Distance = %d km)%n", dest, sb, d.dist[t]);
            total += d.dist[t];
        }
        System.out.println("-----------------------------------------------");
        System.out.println("Total distance for all deliveries: " + total + " km");
    }
}
```

7. RESULTS & DISCUSSION

Test case 1 uses a delivery network of six locations (Warehouse, A, B, C, D, E) connected by nine routes. The shortest-path routes found by the system are highlighted in red in Fig. 2.

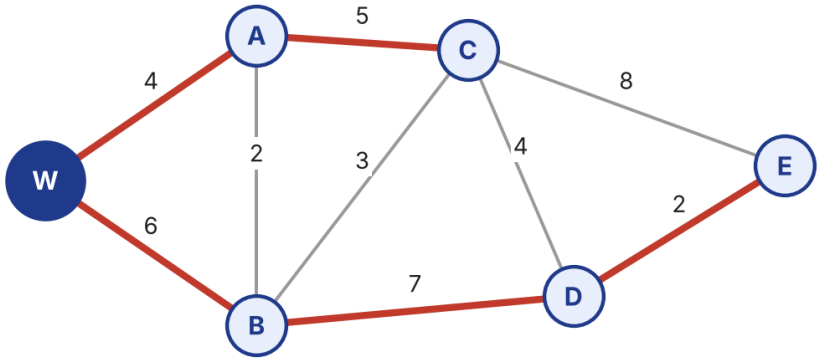


Fig. 2: Sample delivery network (distances in km) with optimized routes highlighted

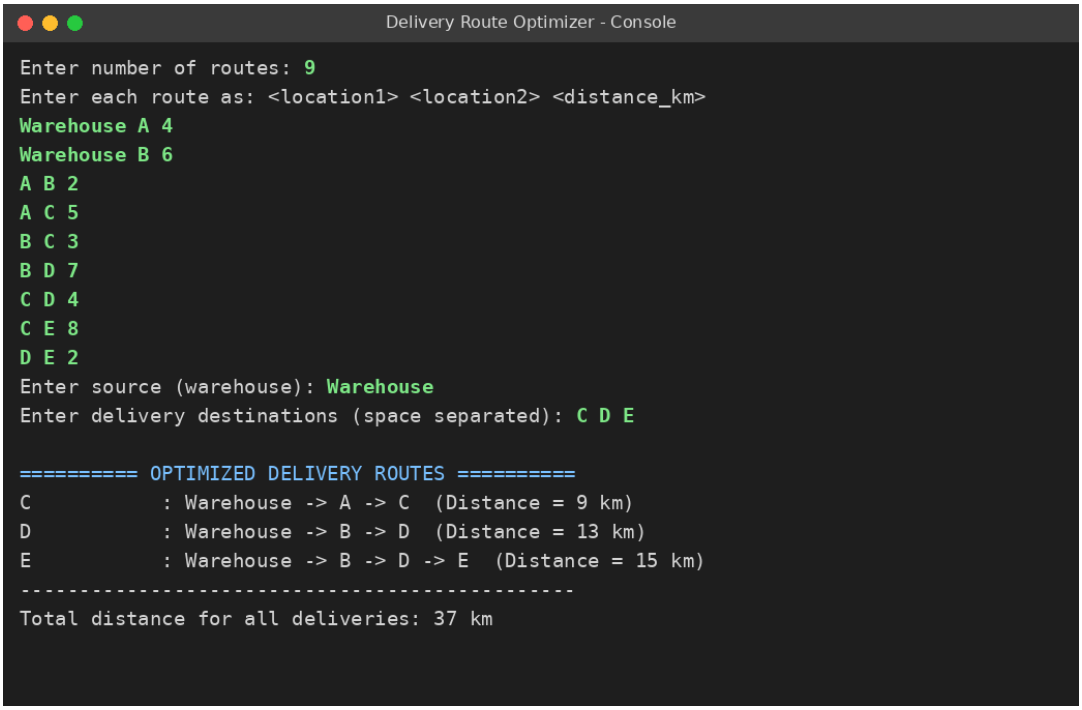


Fig. 3: Console output for test case 1 – routes to C, D and E

The program accepts the routes, source and destinations as input and returns the optimized route to each destination along with the total distance (37 km). The values match a manual shortest-path calculation.

Destination	Optimized route	Distance (km)
C	Warehouse → A → C	9
D	Warehouse → B → D	13
E	Warehouse → B → D → E	15
Total		**37**

Table 1: Results of test case 1

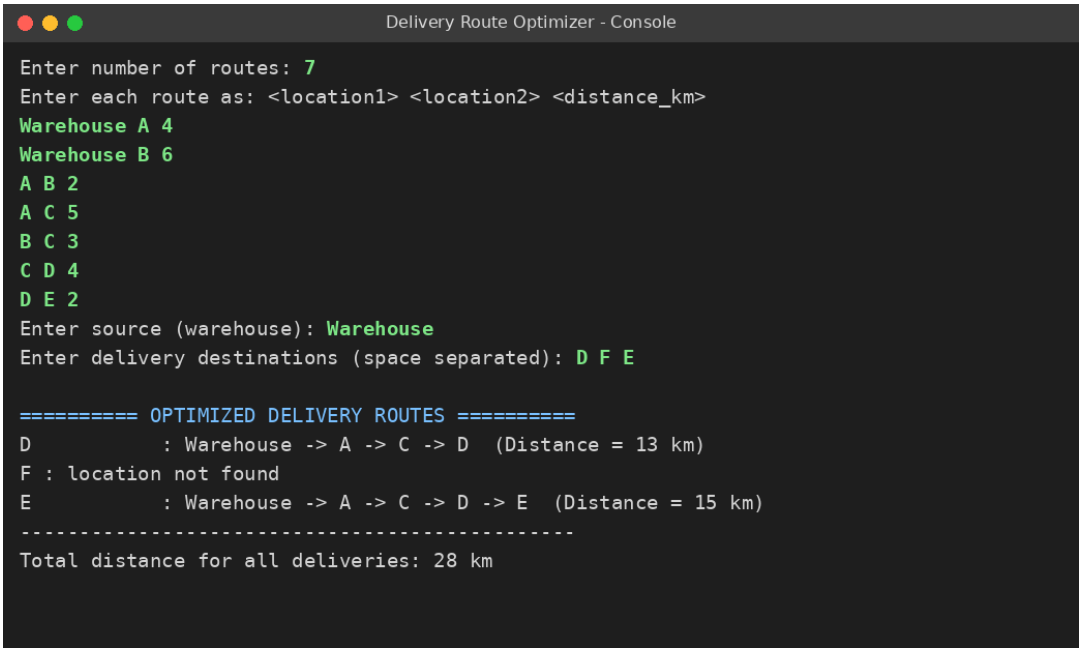


Fig. 4: Console output for test case 2 – an unknown destination is reported and the rest are still routed

In test case 2, the destination "F" does not exist in the network, so the system reports "location not found" and still produces correct routes for the remaining destinations, showing that invalid input is handled gracefully.

Discussion: The optimized route is never longer than a manually chosen one. For destination E in test case 1, a hand-picked route Warehouse → A → C → E covers 17 km, whereas the route found by Dijkstra's Algorithm (Warehouse → B → D → E) covers 15 km, saving about 12% of the distance. Because Dijkstra's Algorithm is executed only once from the source, the routes to all destinations are obtained together, and the interface accepts new locations, distances and destinations dynamically.

8. CONCLUSION & FUTURE SCOPE

The Delivery Route Optimizer successfully applies Data Structures and Algorithms to a real-world logistics problem. The objectives of the project were met: a graph-based model of delivery locations was designed, Dijkstra's Algorithm was implemented with a priority queue, a simple interface accepts locations, distances and destinations, and an optimized route with its total distance is displayed. The system demonstrates the practical use of graphs, priority queues and shortest-path algorithms, helps reduce travel distance, delivery time, fuel consumption and operational cost, and strengthens understanding of algorithm design, graph theory and optimization techniques. It delivers a simple, functional and educational route-management solution.

Future Scope

Near term: Integrate real-time traffic data for dynamic, up-to-date route updates, and implement the A* algorithm for faster, heuristic-based pathfinding.

Medium term: Extend the system to multi-vehicle, multi-destination routing (Vehicle Routing Problem), and add GPS and map API integration for real-world coordinates and visualization.

Long term: Develop a mobile or web application for wider accessibility, and incorporate machine learning to predict demand and traffic patterns.

9. REFERENCES

[1] E. W. Dijkstra, “A note on two problems in connexion with graphs,” Numerische Mathematik, vol. 1, pp. 269–271, 1959.

[2] Dijkstra's Algorithm – Wikipedia. https://en.wikipedia.org/wiki/Dijkstra%27s_algorithm

[3] Dijkstra's Algorithm – GeeksforGeeks. https://www.geeksforgeeks.org/dijkstras-shortest-path-algorithm-greedy-algo-7/

[4] Bellman–Ford Algorithm – Wikipedia. https://en.wikipedia.org/wiki/Bellman%E2%80%93Ford_algorithm

[5] A* Search Algorithm – Wikipedia. https://en.wikipedia.org/wiki/A*_search_algorithm

[6] Vehicle Routing Problem – Wikipedia. https://en.wikipedia.org/wiki/Vehicle_routing_problem

[7] Travelling Salesman Problem – Wikipedia. https://en.wikipedia.org/wiki/Travelling_salesman_problem

[8] Vehicle Routing | OR-Tools | Google for Developers. https://developers.google.com/optimization/routing

[9] PriorityQueue (Java Platform SE 8). https://docs.oracle.com/javase/8/docs/api/java/util/PriorityQueue.html

[10] T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, Introduction to Algorithms, 4th ed., MIT Press, 2022.